

CS162
Operating Systems and
Systems Programming
Lecture 2

Four fundamental abstractions:
thread, address space, process, dual mode

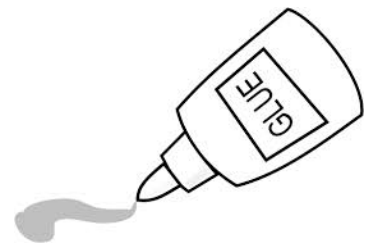
Professor Ion Stoica & Matei Zaharia

<https://cs162.org/>

Recall: What is an Operating System?

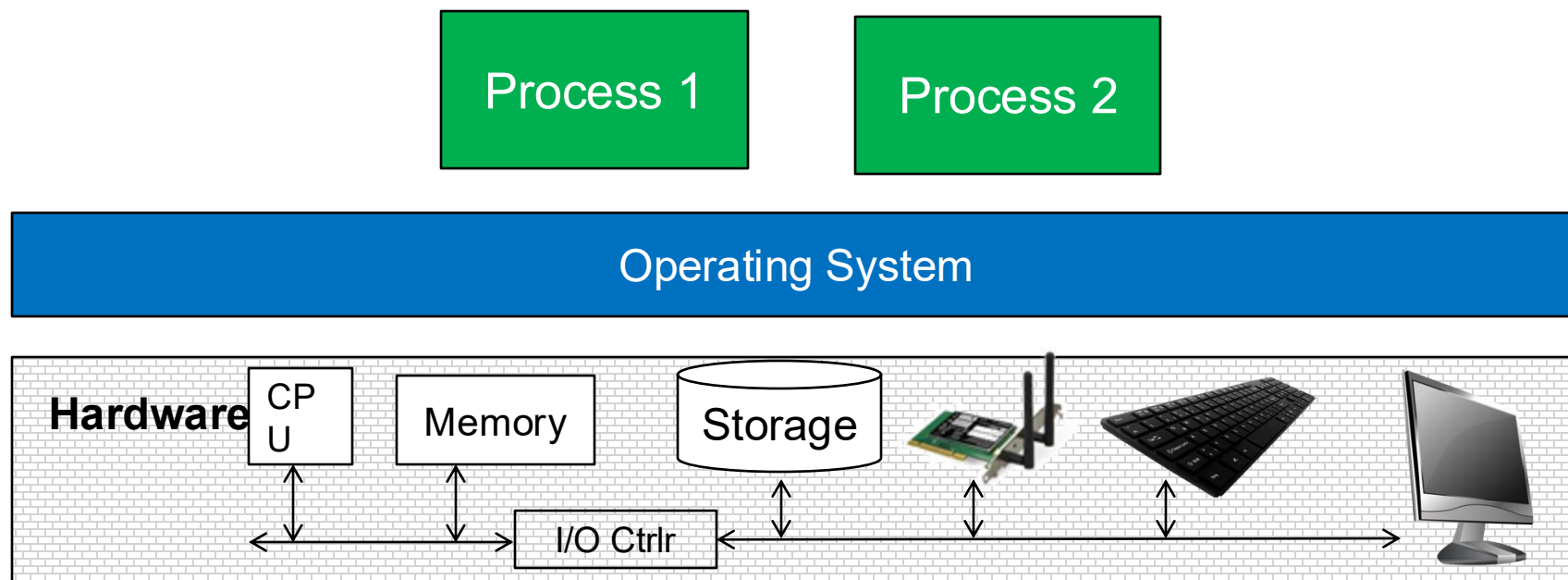


- Referee
 - Manage protection, isolation, and sharing of resources
 - » Resource allocation and communication
- Illusionist
 - Provide clean, easy-to-use abstractions of physical resources
 - » Infinite memory, dedicated machine
 - » Higher level objects: files, users, messages
 - » Masking limitations, virtualization
- Glue
 - Common services
 - » Storage, Window system, Networking
 - » Sharing, Authorization
 - » Look and feel



Virtualization: Operating System

Process: an active instance of a program

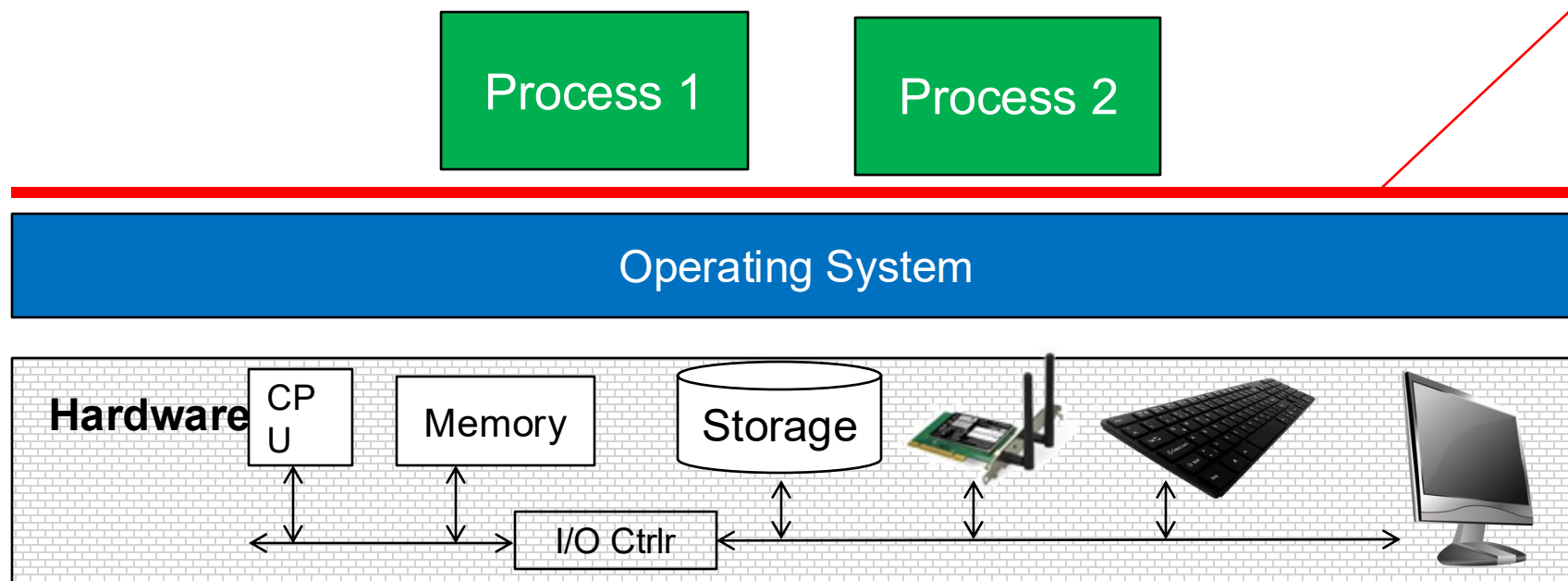


Operating System virtualizes* hardware to processes (applications) providing high-level abstractions (e.g., process thread, sockets, etc)

***virtualization: provides the illusion that each process uses its own machine; also provides illusion of infinite memory and processor (though there will be a hit in performance)**

Virtualization: Operating System

Process: an active instance of a program

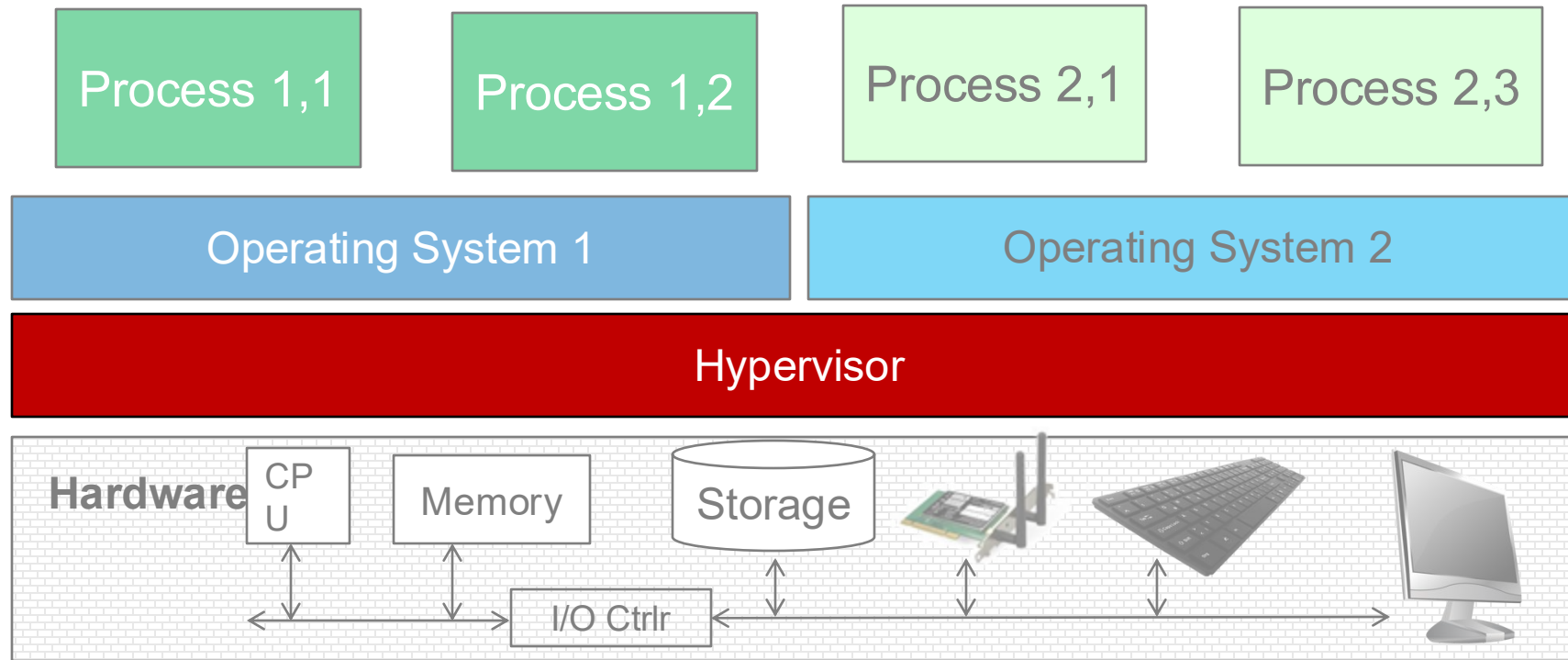


OS as Illusionist:
Hides hardware complexity;
Virtualizes hardware, e.g.,
provides clean abstractions
like “infinite” CPU and compu

**Operating System
virtualizes* hardware
to processes (applications)
providing high-level
abstractions (e.g., process
thread, sockets, etc)**

***virtualization: provides the illusion that each process uses its own machine; also provides illusion of infinite memory and processor (though there will be hit in performance)**

Virtualization: Virtual Machine

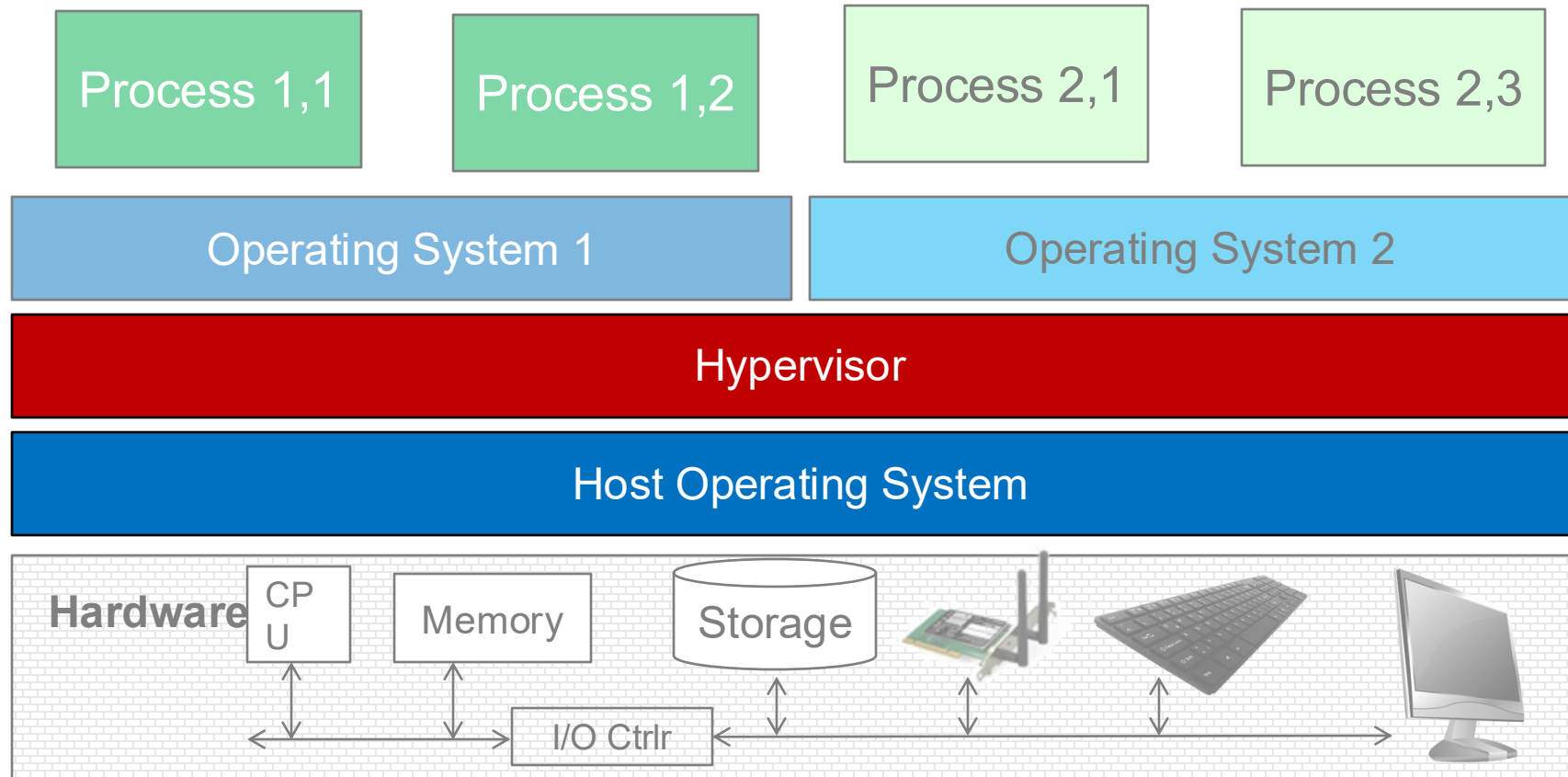


Hypervisor creates virtual machines. A virtual machine virtualizes* hardware to Operating System providing same hardware abstraction



***virtualization: provides the illusion that each operating system is using the same physical machine; It provides fault isolation; if an OS crashes won't affect other OSES**

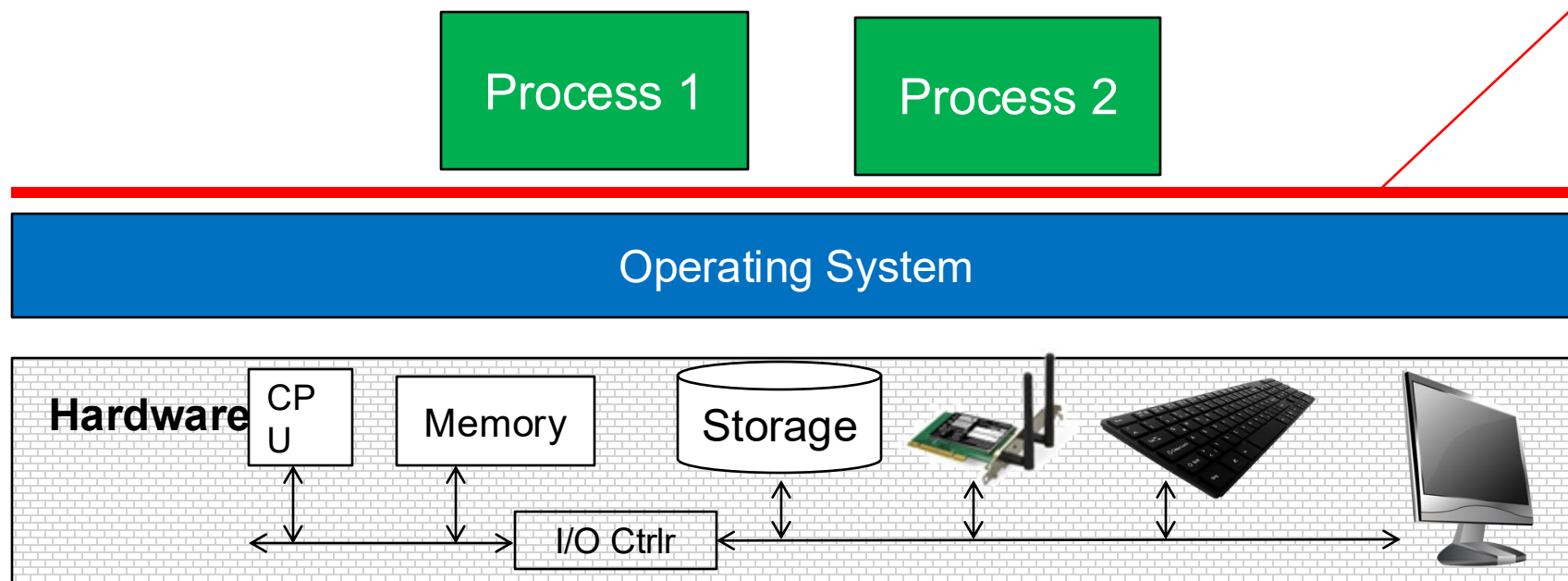
Virtualization: Virtual Machine



Virtual machines can be implemented on top of an existing OS as well (easier to deploy; provide OS isolation; it makes developing and testing for different OSes easier)

Virtualization: Operating System

Process: an active instance of a program



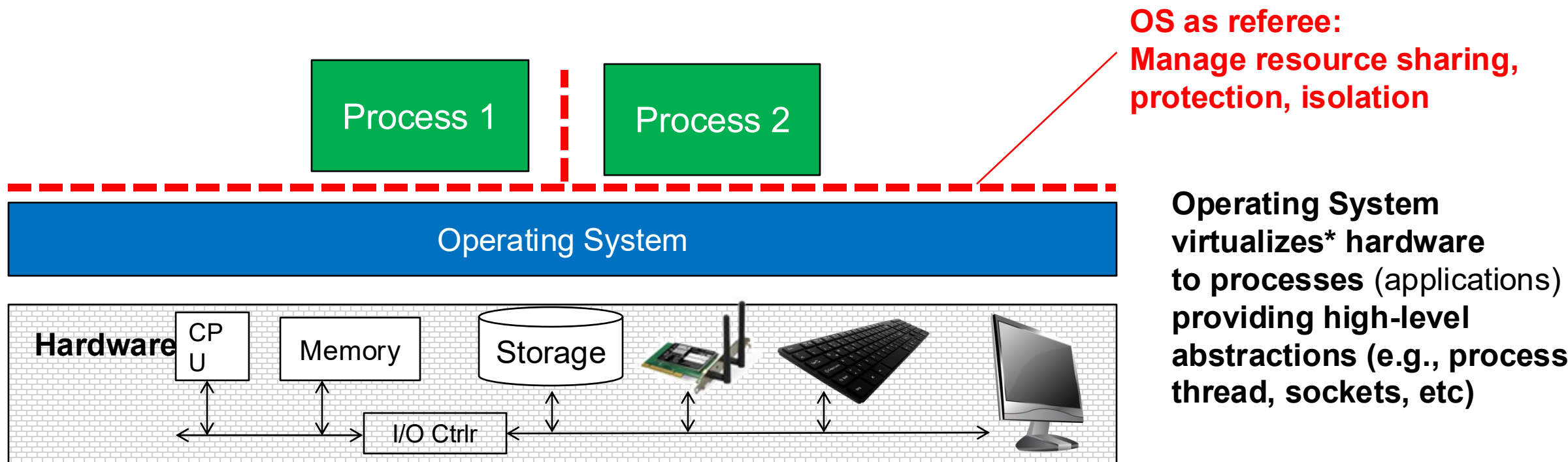
OS as glue:
Provides higher level services to make programmer's job easier, e.g., file systems, networking, windows

Operating System virtualizes* hardware to processes (applications) providing high-level abstractions (e.g., process thread, sockets, etc)

***virtualization: provides the illusion that each process uses its own machine; also provides illusion of infinite memory and processor (though there will be a hit in performance)**

Virtualization: Operating System

Process: an active instance of a program

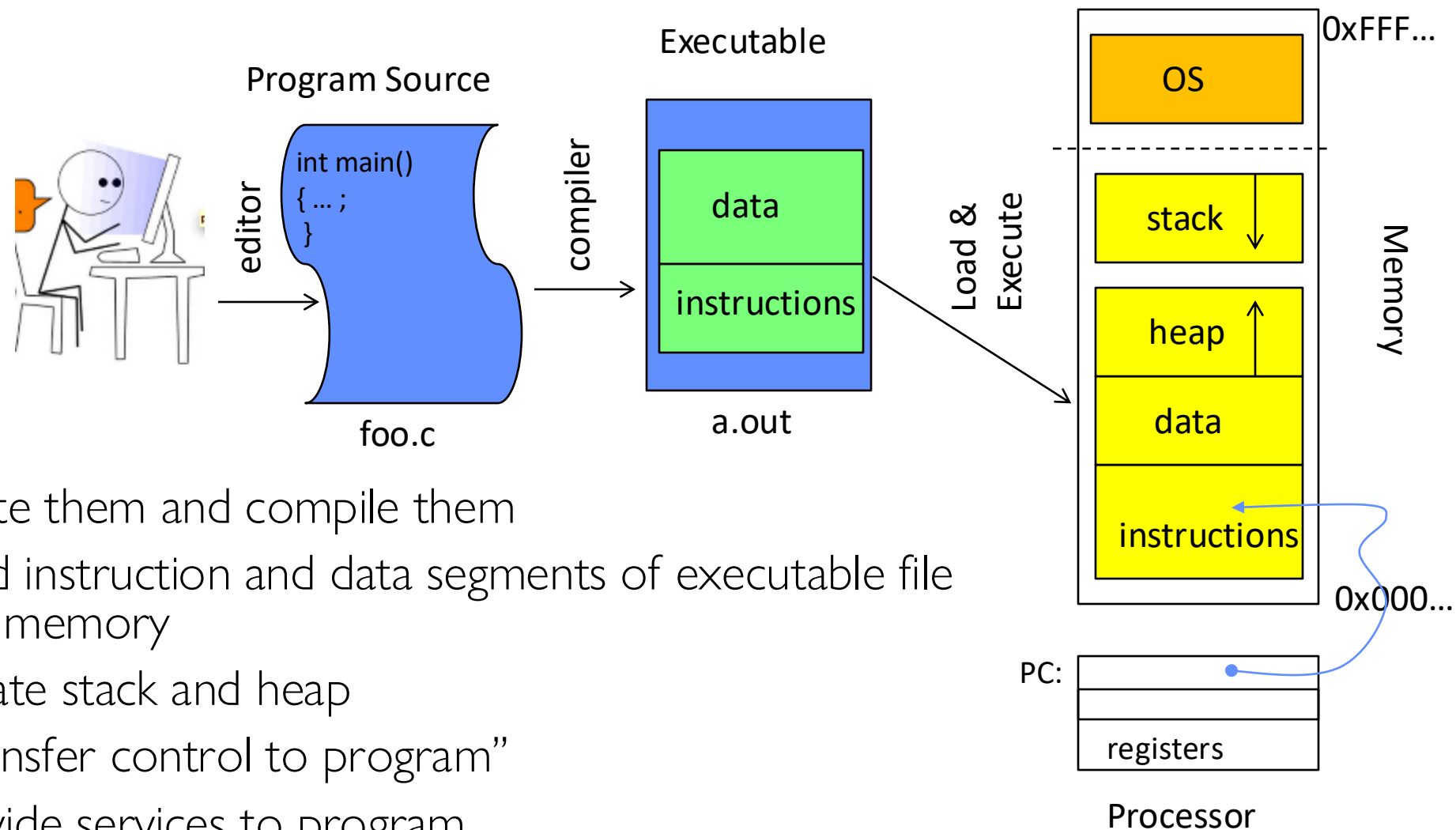


***virtualization: provides the illusion that each process uses its own machine; also provides illusion of infinite memory and processor (though there will be a hit in performance)**

Today: Four Fundamental OS Concepts

- **Process:** an instance of a running program
 - Address Space + One or more Threads
- **Thread: Execution Context**
 - Fully describes program state
 - Program Counter, Registers, Stack, Execution Flags,
- **Address space** (with or w/o **translation**)
 - Set of memory addresses accessible to program (for read or write)
 - May be distinct from memory space of the physical machine (in which case programs operate in a virtual address space)
- **Dual mode operation / Protection**
 - Only the “system” has the ability to access certain resources
 - Combined with translation, isolates programs from each other and the OS from programs

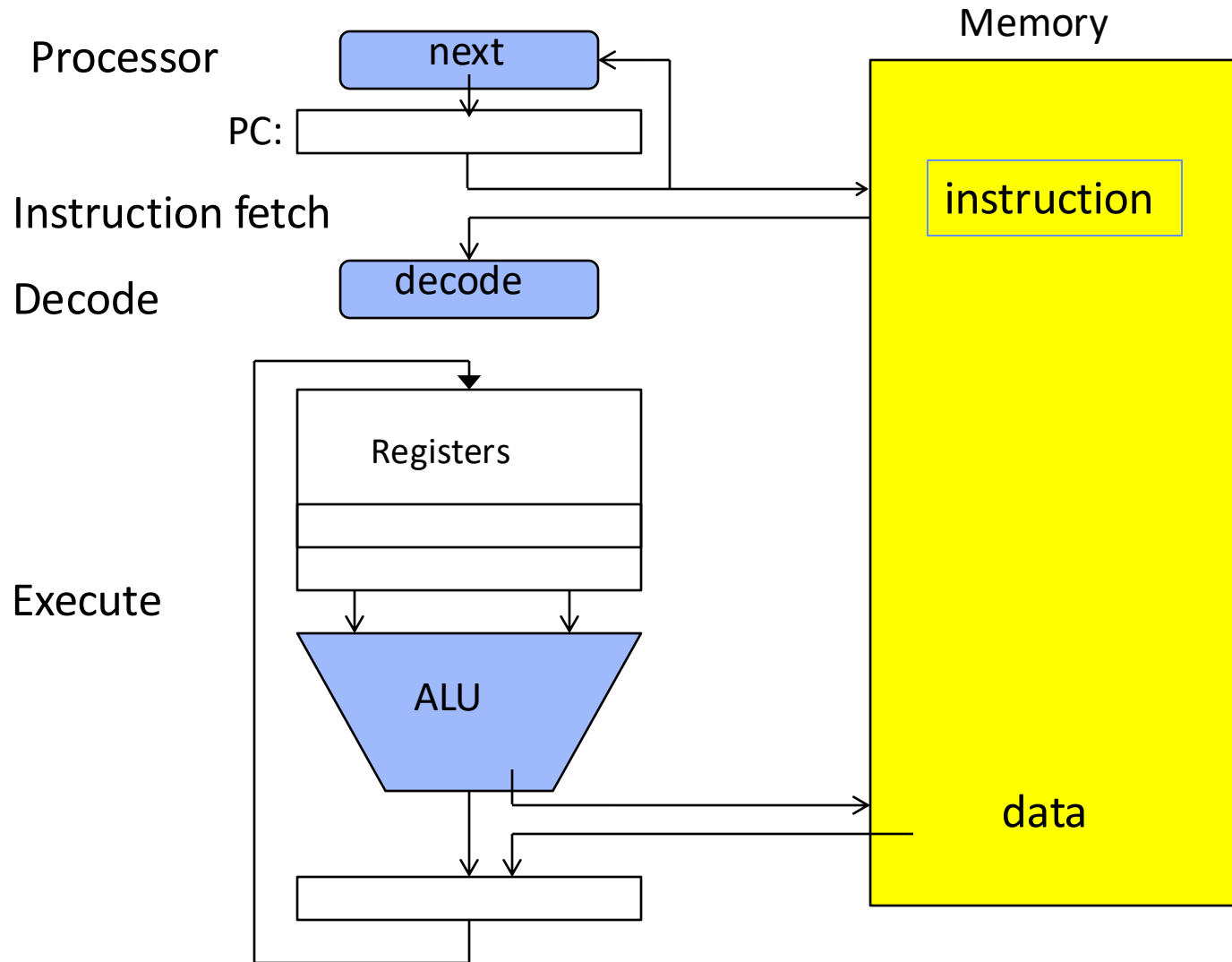
OS Bottom Line: Run Programs



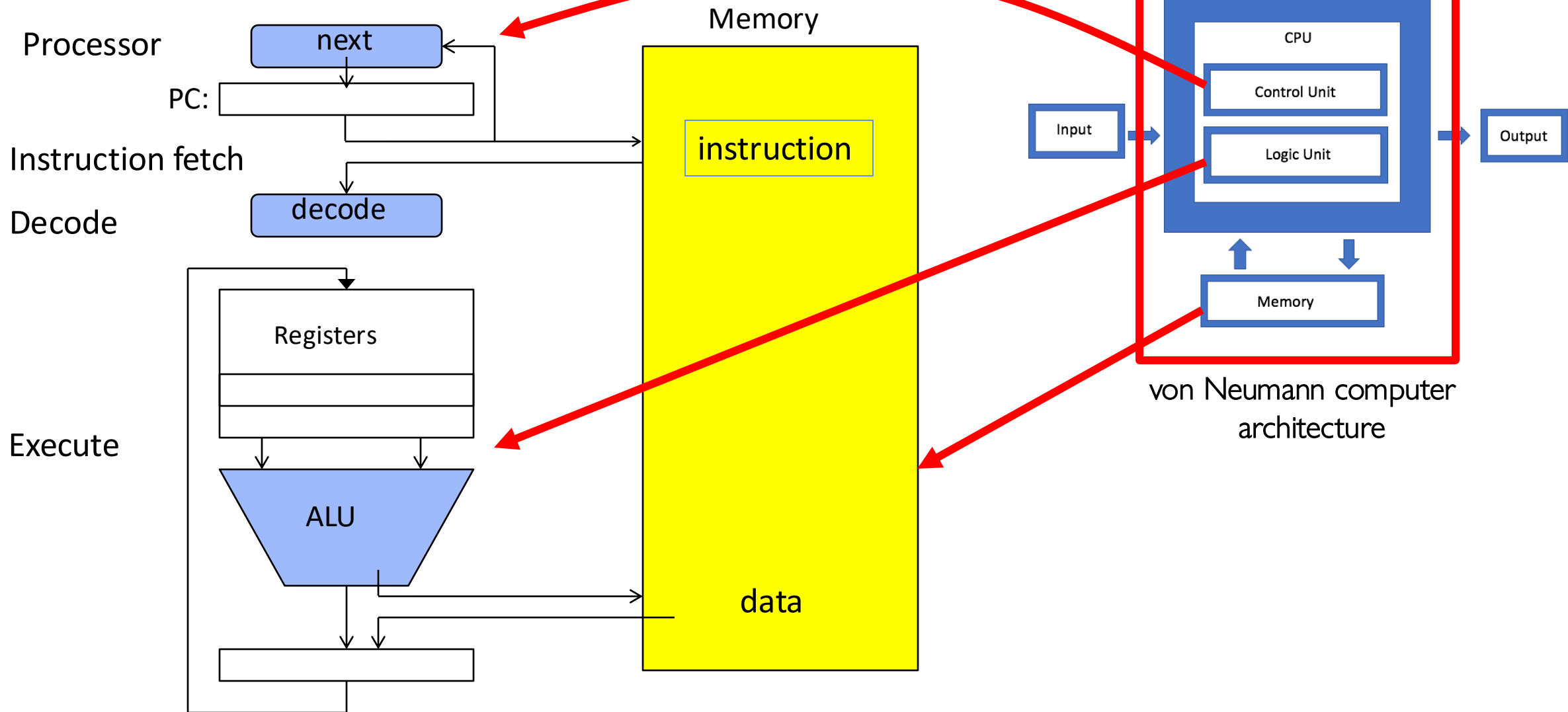
- Write them and compile them
- Load instruction and data segments of executable file into memory
- Create stack and heap
- “Transfer control to program”
- Provide services to program
- While protecting OS and program

Recall (61C): Instruction Fetch/Decode/Execute

The instruction cycle



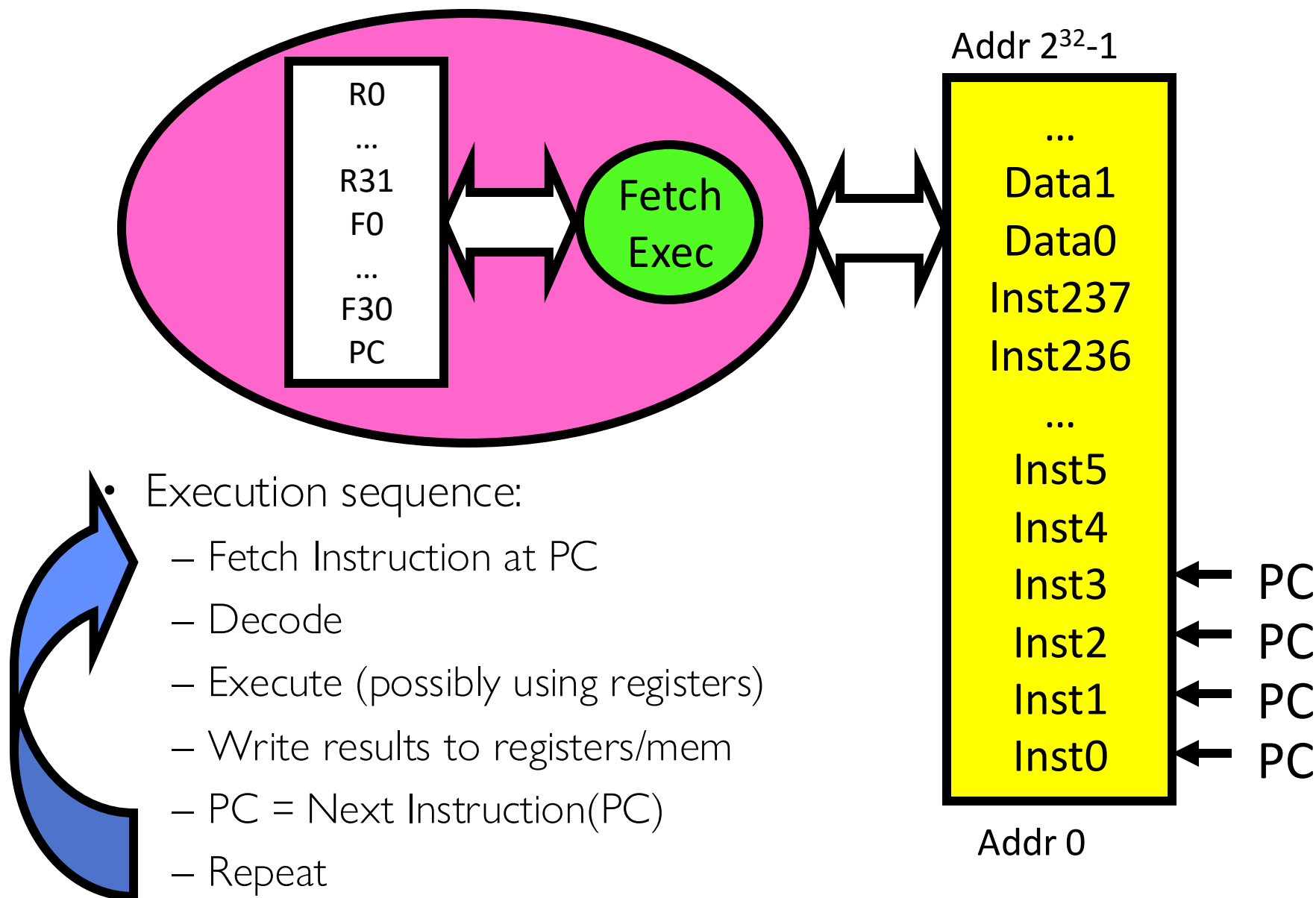
The instruction cycle



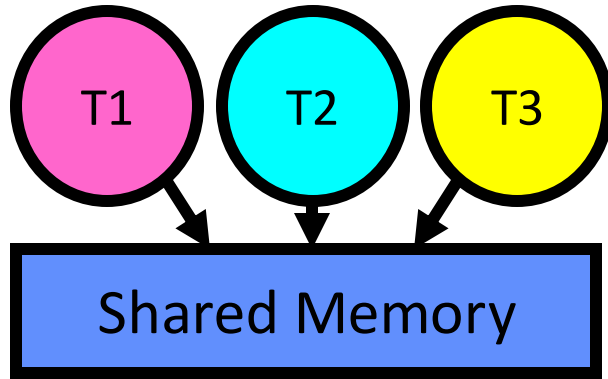
First OS Concept: Thread of Control

- **Thread:** Single unique execution context
 - Program Counter, Registers, Execution Flags, Stack, Memory State
- A thread is *executing* on a processor (core) when it is *resident* in the processor registers
- Resident means: Registers hold the root state (context) of the thread:
 - Including program counter (PC) register & currently executing instruction
 - » PC points at next instruction *in memory*
 - » Instructions stored in memory
 - Including intermediate values for ongoing computations
 - » Can include actual values (like integers) or pointers to values *in memory*
 - Stack pointer holds the address of the top of stack (which is *in memory*)
 - The rest is “in memory”
- A thread is *suspended* (not *executing*) when its state *is not* loaded (resident) into the processor
 - Processor state pointing at some other thread
 - Program counter register *is not* pointing at next instruction from this thread
 - Often: a copy of the last value for each register stored in memory

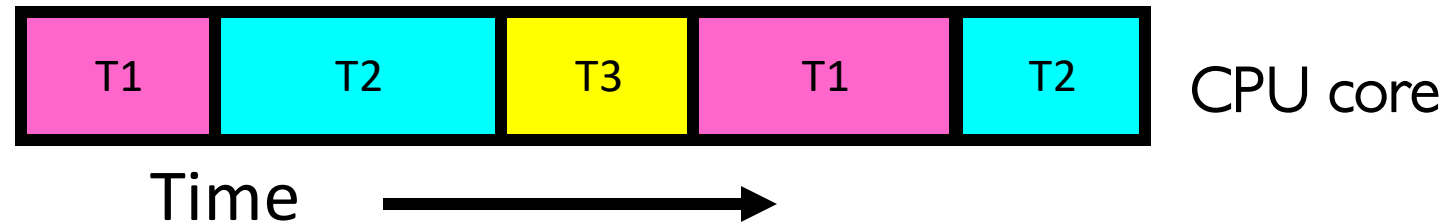
Recall (61C): What happens during program execution?



Illusion of Multiple Processors with single core

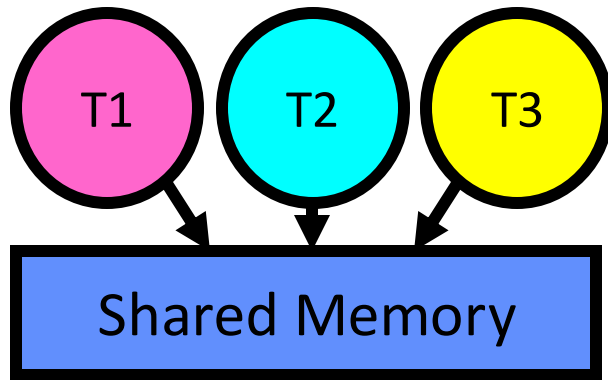


Programmer's View

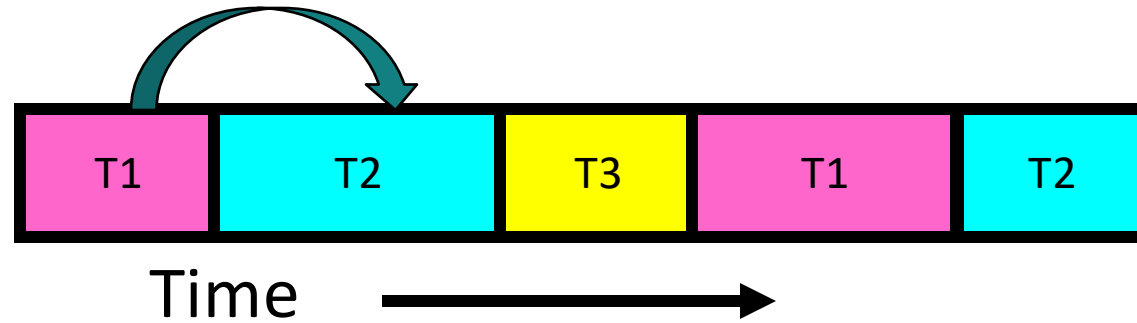


- Assume a **single processor (core)**. How do we provide the illusion of multiple processors?
 - Mux in time!
- Content of a thread:
 - Program counter
 - Stack pointer
 - Registers
- Where is the content?
 - On the real (physical) core, or
 - Saved in chunk of memory – called the **Thread Control Block (TCB)**

Illusion of Multiple Processors (Continued)



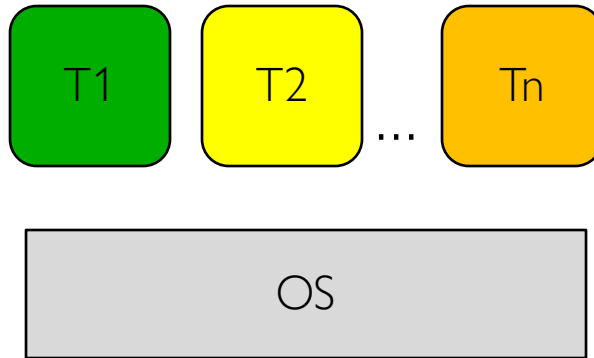
Programmer's View



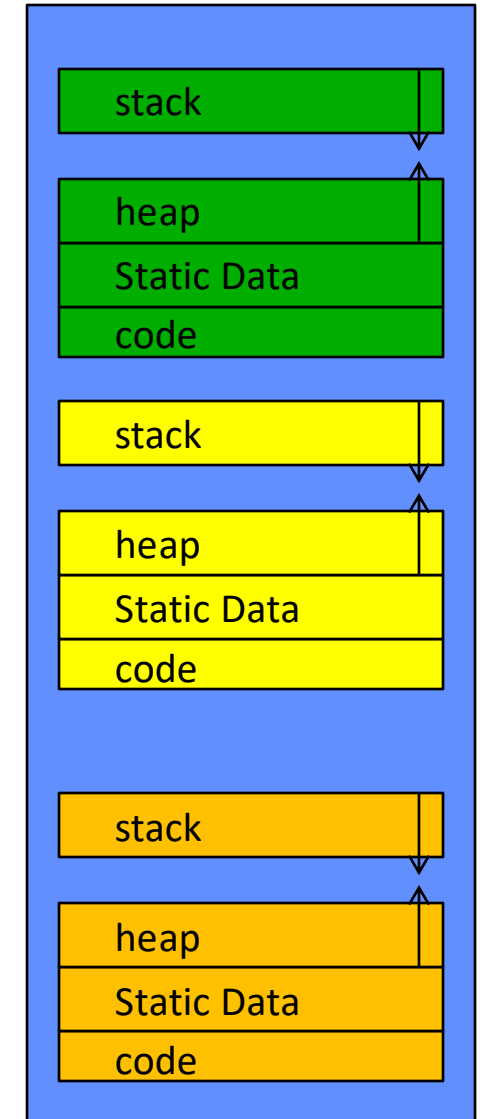
CPU core

- What happens on a switch?
 - Run OS [how?]
 - Save PC, SP, ... in T1's TCB (memory)
 - Load PC, SP, ... from T2's TCB, jumped to PC
- What triggered this switch?
 - Timer, voluntary yield, I/O, other things we will discuss

Multiprogramming - Multiple Threads of Control

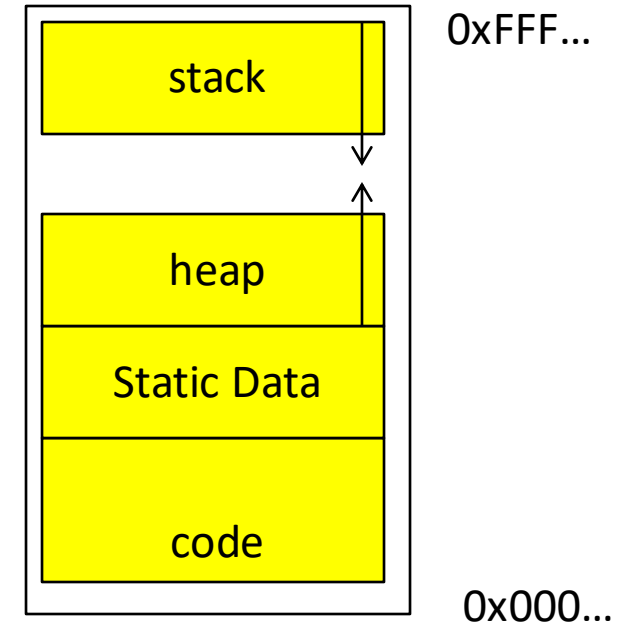


- Thread Control Block (TCB)
 - Holds contents of registers when thread not running
 - What other information?
- Where are TCBs stored?
 - For now, in the kernel
- PINTOS? – read `thread.h` and `thread.c`

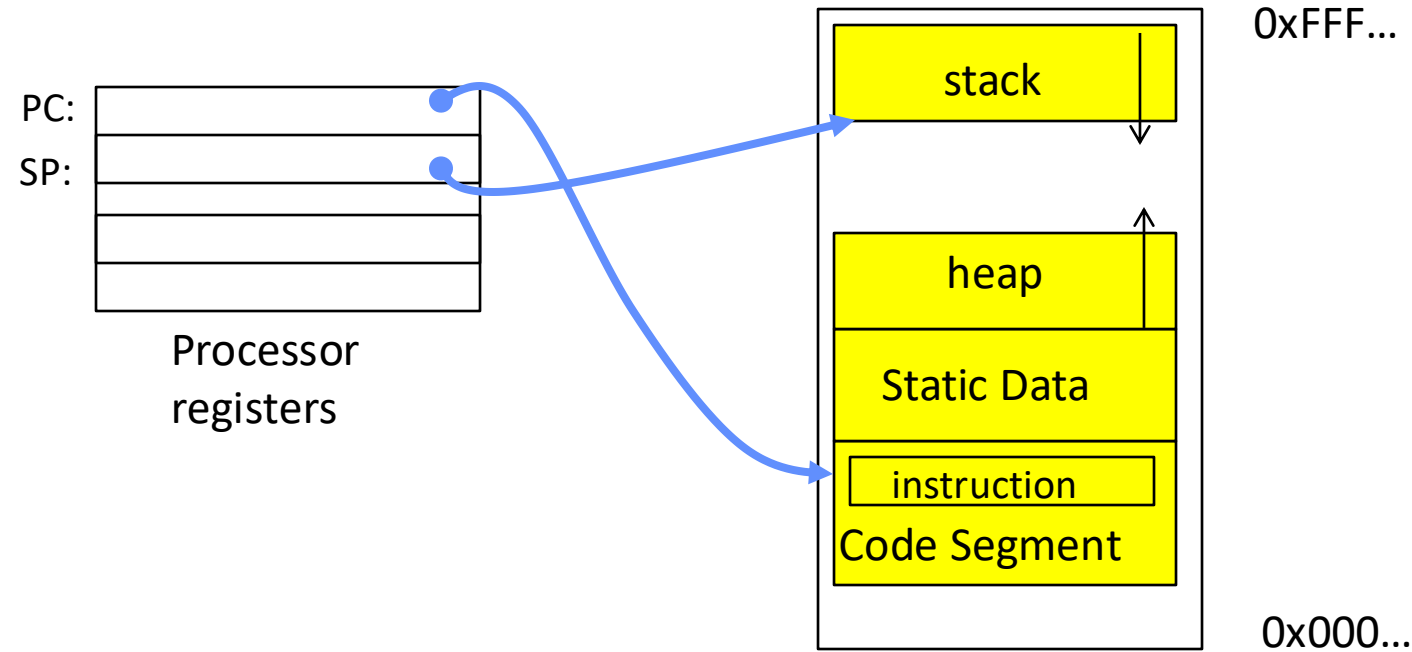


Second OS Concept: Address Space

- Address space $\Rightarrow$ the set of accessible addresses + state associated with them:
 - For 32-bit processor: $2^{32} = 4$ billion (10^9) addresses
 - For 64-bit processor: $2^{64} = 18$ quintillion (10^{18}) addresses
- What happens when you read or write to an address?
 - Perhaps acts like regular memory
 - Perhaps ignores writes
 - Perhaps causes I/O operation
 - » (Memory-mapped I/O)
 - Perhaps causes exception (fault)
 - Communicates with another program
 -



Address Space: In a Picture



- What's in the **code segment**? Static **data segment**?
- What's in the **stack segment**?
 - How is it allocated? How big is it?
- What's in the **heap segment**?
 - How is it allocated? How big?

Simple Example

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;

void stack_function() {
    int stack_var_in_func = 30;
    printf("Address of stack_var_in_func: %p\n", (void*)&stack_var_in_func);
}

int main() {
    int stack_var = 20;

    int *heap_var = (int*)malloc(sizeof(int));
    if (heap_var == NULL) {
        fprintf(stderr, "Failed to allocate memory on the heap\n");
        return 1;
    }
    *heap_var = 40; // Storing a value in the heap memory.

    // The programmer is responsible for freeing heap memory.
    free(heap_var);

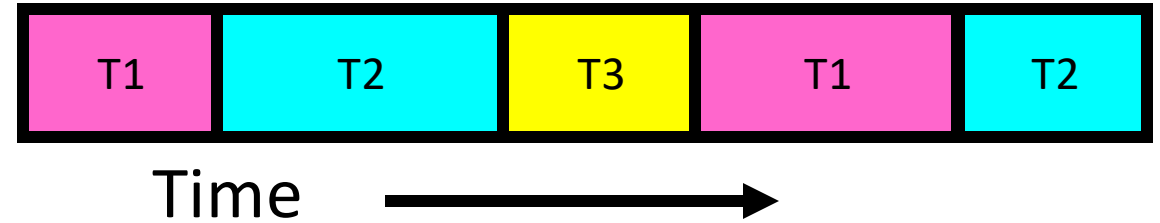
    return 0;
}
```

Diagram illustrating memory segments:

- Data Segment**: Points to the global variable `global_var = 10;`
- Stack Segment**: Points to the local variables `stack_var_in_func = 30;` and `stack_var = 20;`
- Heap**: Points to the dynamically allocated memory `heap_var = (int*)malloc(sizeof(int));`

Previous discussion of threads: Very Simple Multiprogramming

- All T's share non-CPU resources
 - Memory, I/O Devices
- Each thread can read/write memory
 - Perhaps data of others
 - Can overwrite OS ?
- Unusable?
- This approach is used in
 - Very early days of computing
 - Embedded applications
 - MacOS 1-9/Windows 3.1 (switch only with voluntary yield)
- However, it is risky...



Super Mario Land 2

Mario could exit a level and explore the entire memory of the system!



<https://youtu.be/FPzuYWbnln4>

Simple Multiplexing has no Protection!

- Operating System must protect itself from user programs
 - Reliability: compromising the operating system generally causes it to crash
 - Security: limit the scope of what threads can do
 - Privacy: limit each thread to the data it is permitted to access
 - Fairness: each thread should be limited to its appropriate share of system resources (CPU time, memory, I/O, etc)
- OS must protect User programs from one another
 - Prevent threads owned by one user from impacting threads owned by another user
 - Example: prevent one user from stealing secret information from another user

Administrivia: Getting started

- Should be working on Homework 0 already! $\Rightarrow$ Due Friday (9/5)
 - cs162-xx account, Github account, registration survey
 - VM environment for the course
 - Get familiar with all the cs162 tools, submitting to autograder via git
- Start Project 0 Wednesday (9/3) !
 - To be done on your own – like a homework
- Reminder of Resources for you (look at Resources tab on homepage!)
 - Quick tutorial about C: Ladder
 - Oreilly Books on: C language, Git environment, Rust (for later)
- Slip days: I'd bank these and not spend them right away!
 - 24h grace period for each deadline (homeworks and projects)
 - You have 5 slip days for homework
 - You have 5 slip days for projects

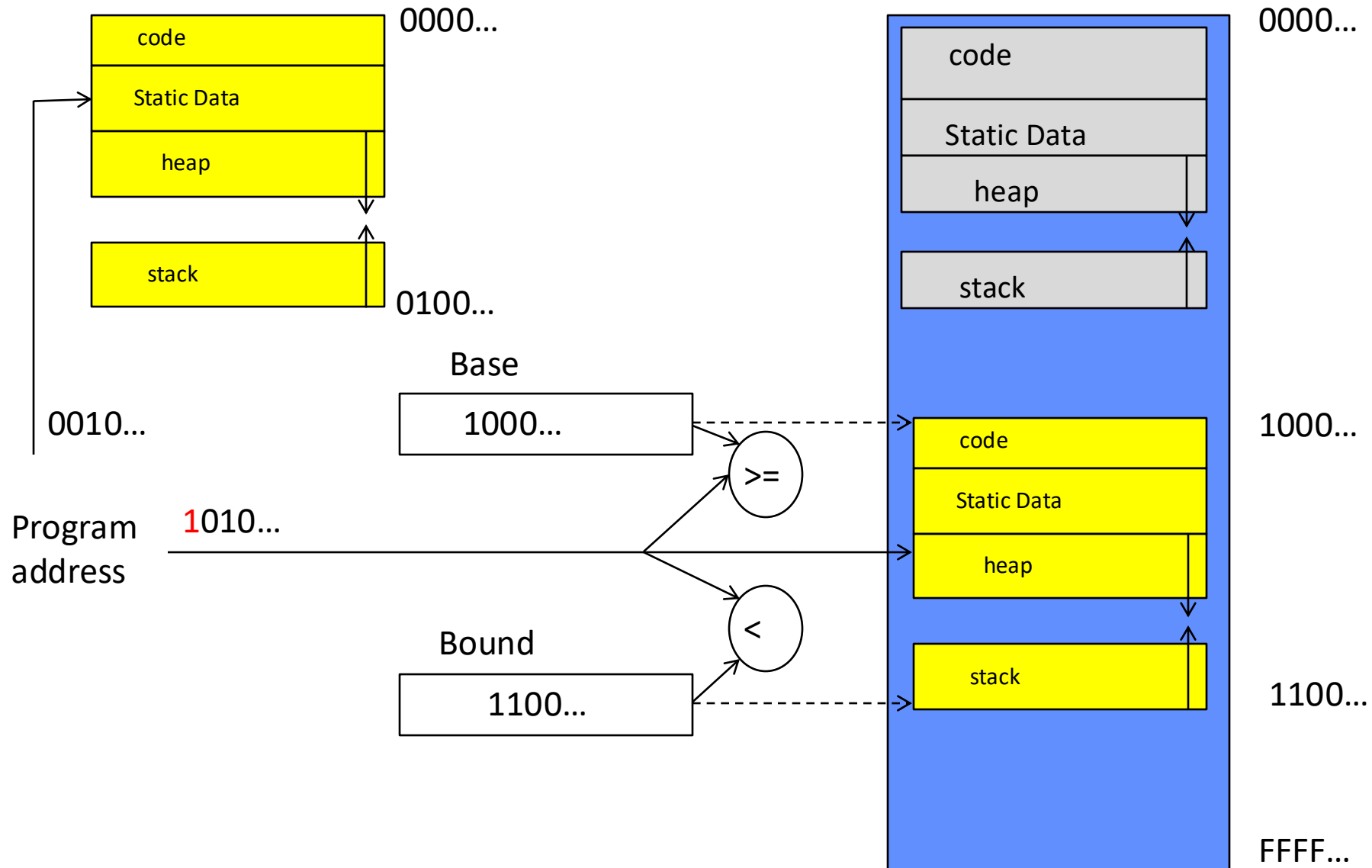
Administrivia (Con't)

- Please don't use slip days on Homework 0 / Project 0!
 - You will start off on wrong foot!
- Class size is fixed at 240 students; 8 discussion sections
 - Will be moving more students from waitlist $\Rightarrow$ class as students drop
 - If you are on waitlist, *assume you could get into the class at any time in next week or so!*
 - Keep up with work (until you drop from the class, or we close the class)
 - » If cannot get credentials for autograder, contact us (I believe we have an Ed thread for that!)
- Friday (9/5) is drop day for this class!
 - Very hard to drop afterwards...
 - Please drop sooner if you are going to anyway $\Rightarrow$ Let someone else in!

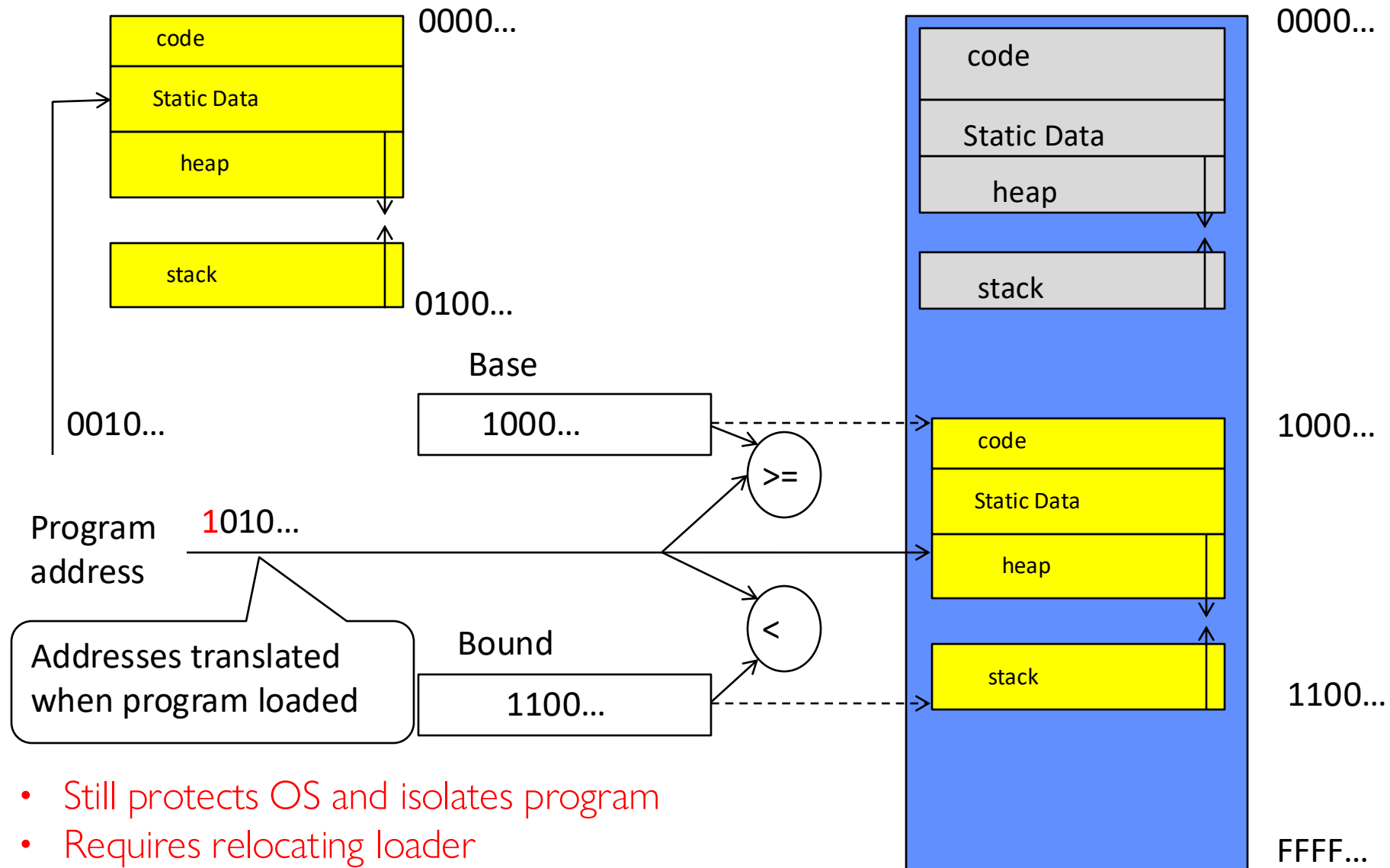
What can the hardware do to help the OS
protect itself from programs?

How about: Limit the *ACCESS* that threads have to memory

Simple Protection: Base and Bound (B&B)

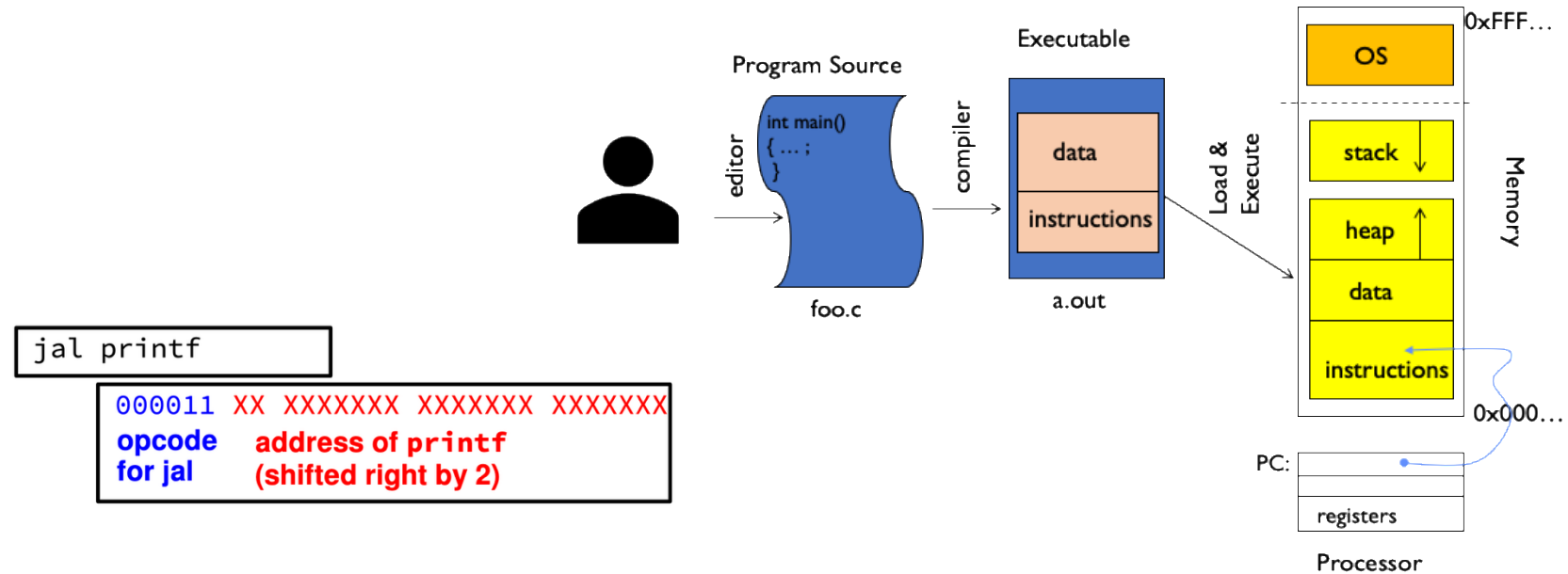


Simple Protection: Base and Bound (B&B)



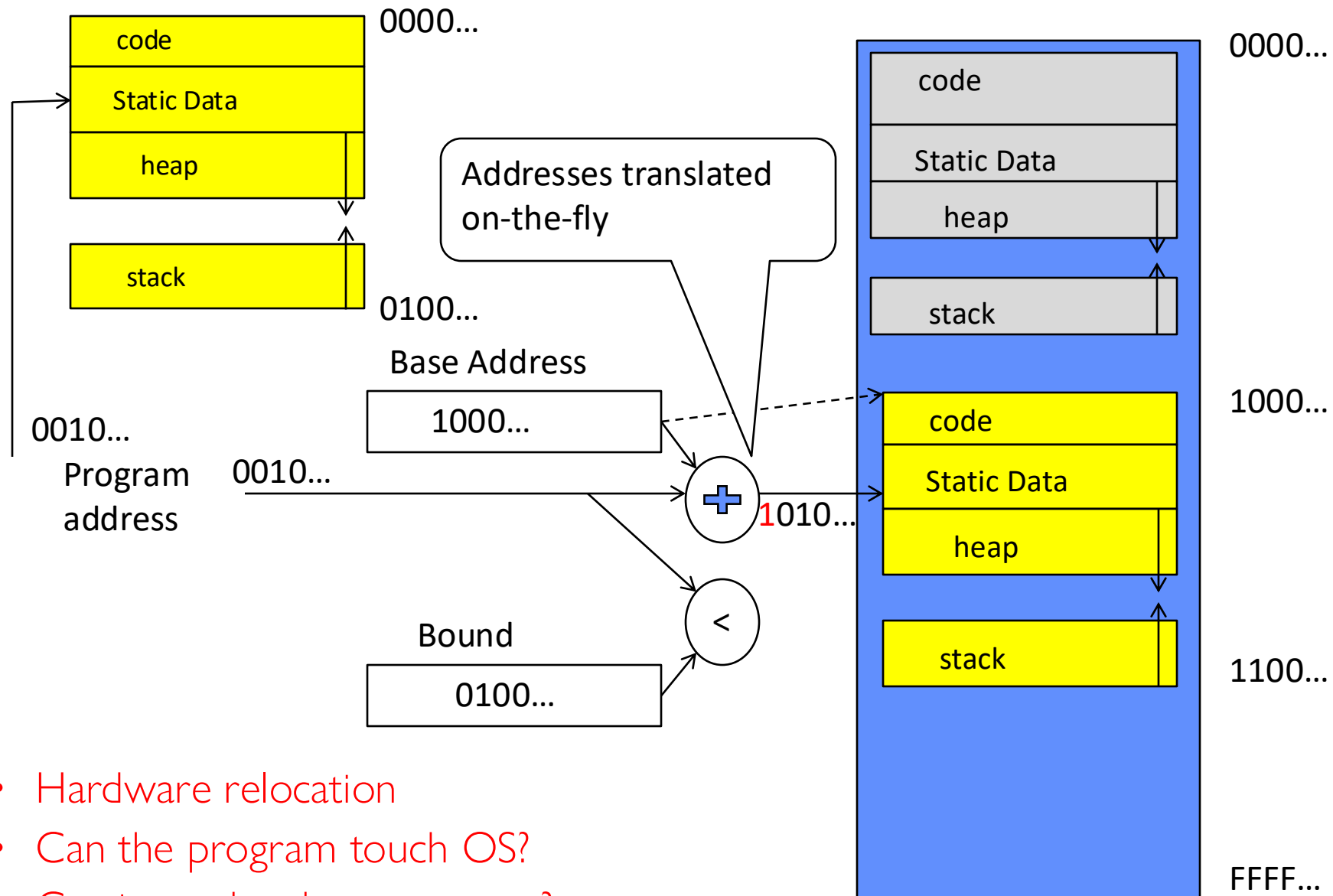
- Still protects OS and isolates program
- Requires relocating loader
- No addition on address path

61C Review: Relocation



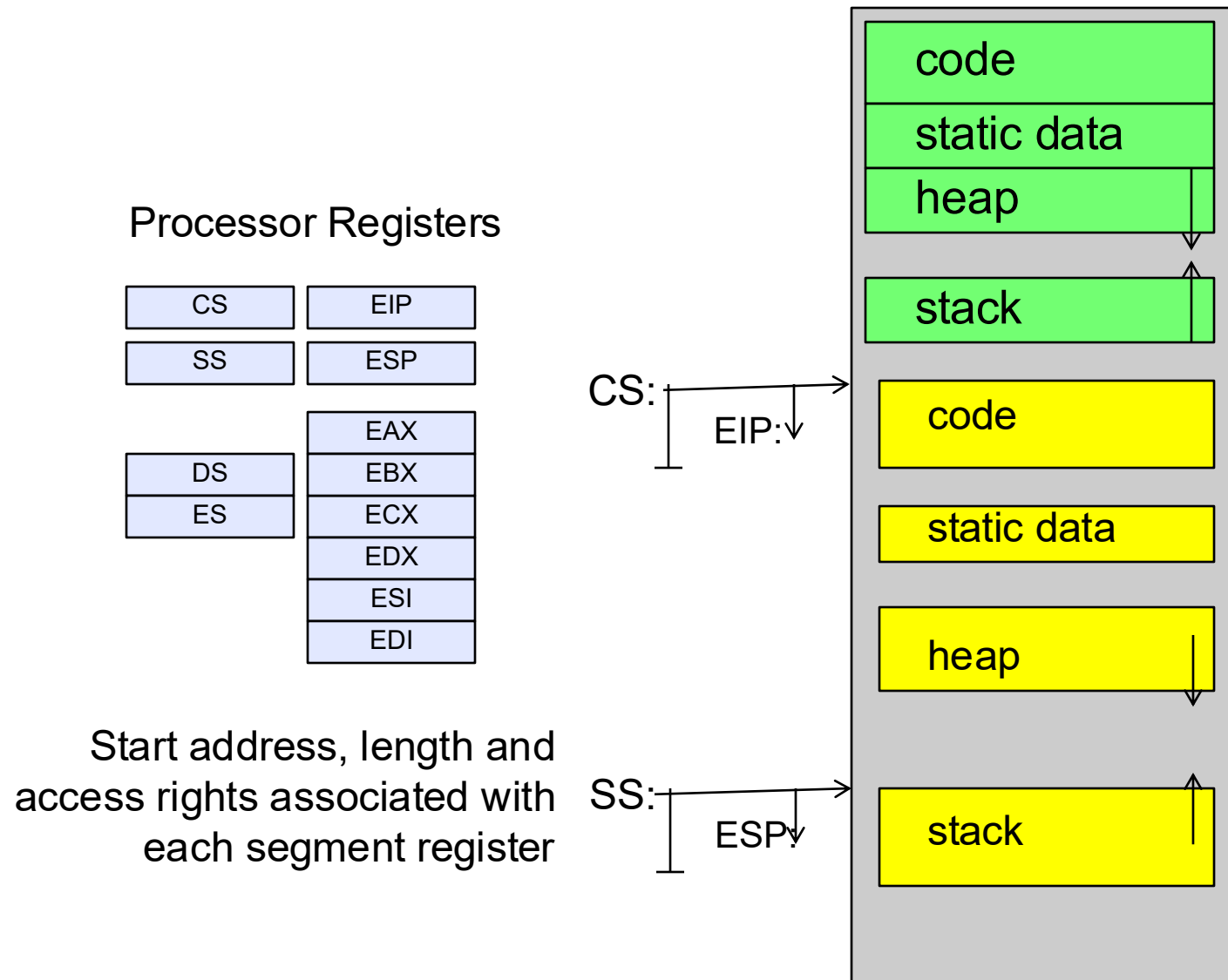
- Compiled .obj file linked together in an .exe
- All address in the .exe are as if it were loaded at memory address 00000000
- File contains a list of all the addresses that need to be adjusted when it is “relocated” to somewhere else.

Simple address translation with Base and Bound



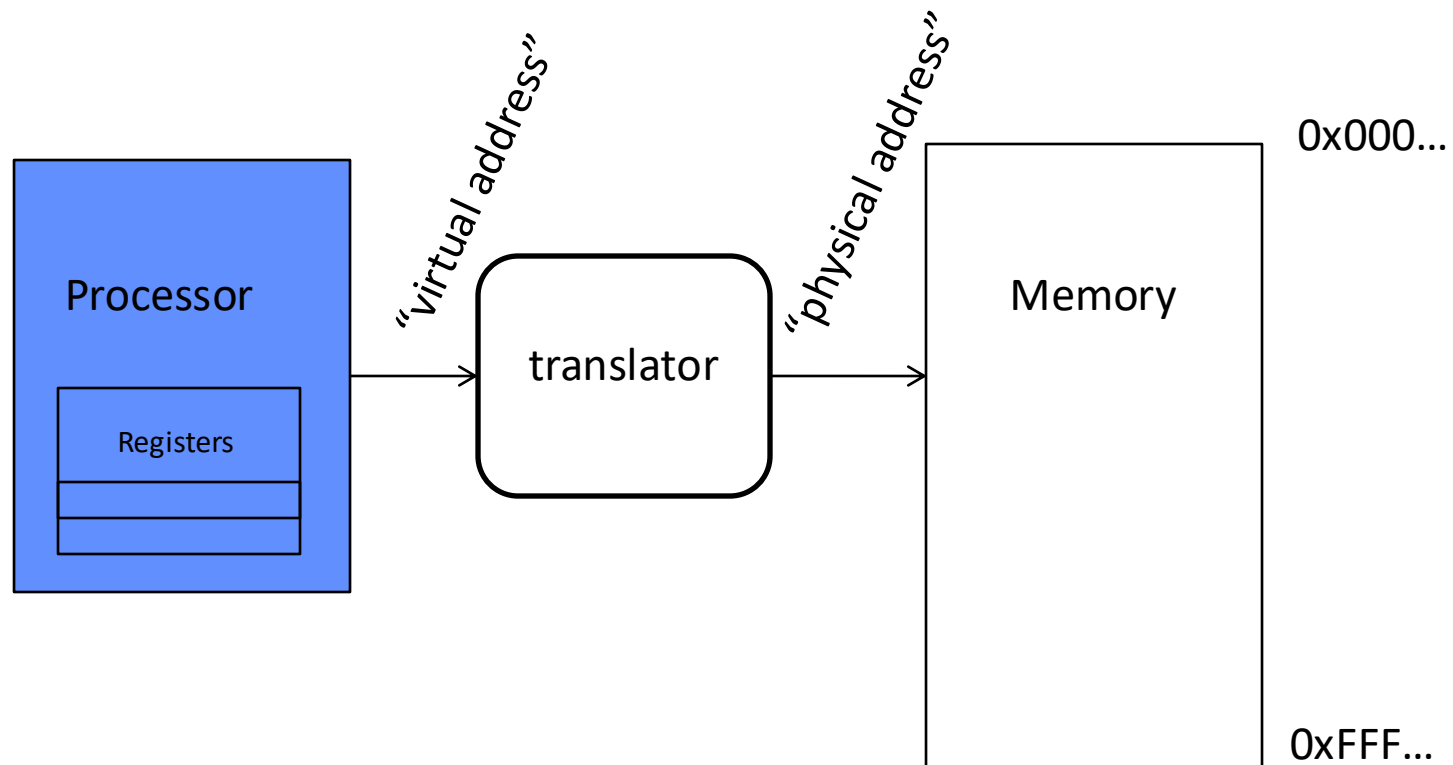
- Hardware relocation
- Can the program touch OS?
- Can it touch other programs?

x86 – segments and stacks



Another idea: Address Space Translation

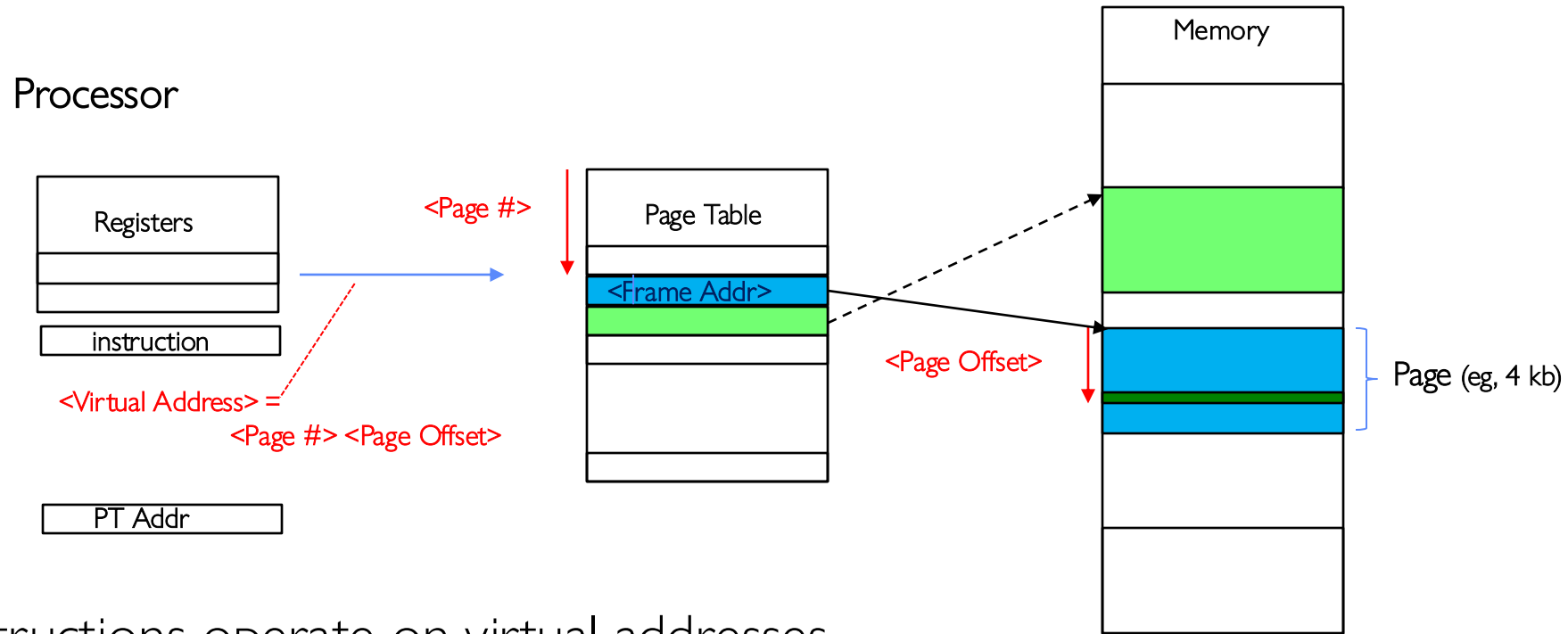
- Program operates in an address space that is distinct from the physical memory space of the machine



Paged Virtual Address Space

- What if we break the entire virtual address space into equal size chunks (i.e., pages) have a base for each?
- All pages same size, so easy to place each page in memory!
- Hardware translates address using a **page table**
 - Each page has a separate base
 - The “bound” is the page size
 - Special hardware register stores pointer to page table
 - Treat memory as page size frames and put any page into any frame ...
- Another cs61C review...

Paged Virtual Address (from 61C)

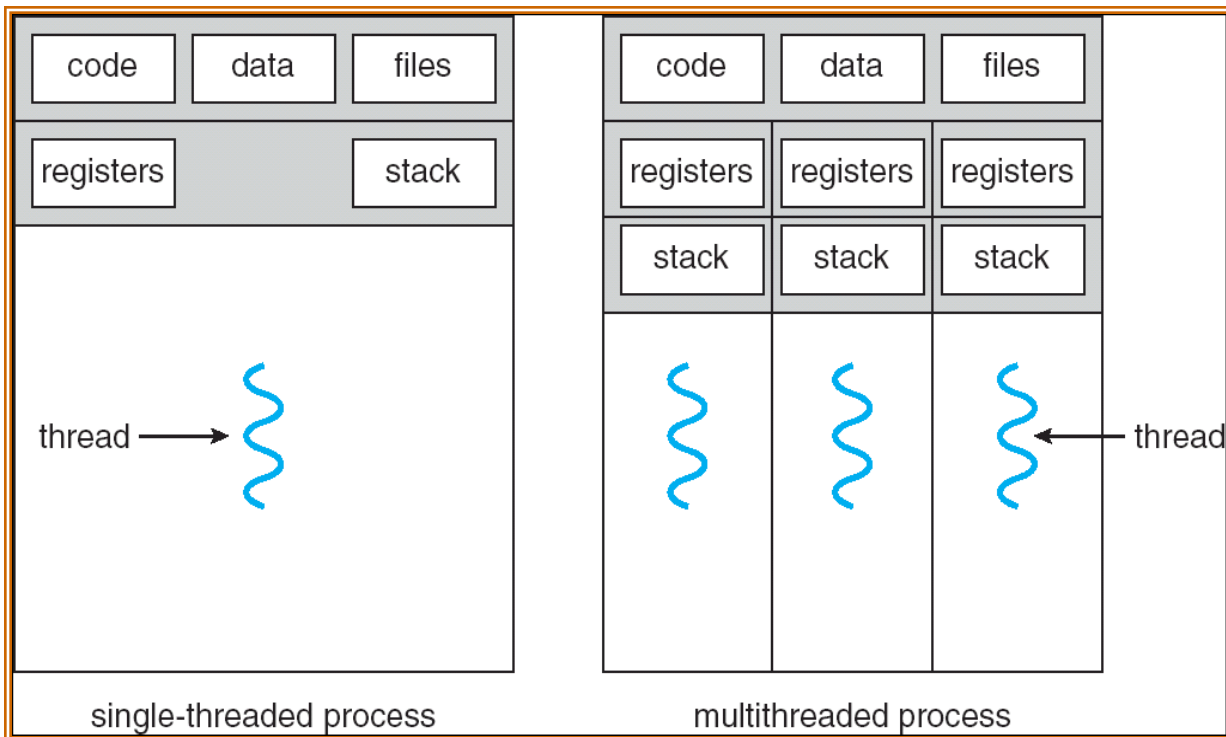


- Instructions operate on virtual addresses
 - Instruction address, load/store data address
- Translated to a physical address through a Page Table by the hardware
- Any Page of address space can be in any (page sized) frame in memory
 - Or not-present (access generates a page fault)
- Special register holds page table base address (of the process)

Third OS Concept: Process

- **Definition:** execution environment for a program with Restricted Rights
 - Owns memory (address space)
 - Owns file descriptors, file system context, ...
 - Encapsulate one or more threads sharing process resources
- Application program executes as a process
 - Complex applications can fork/exec child processes [later!]
- Why **processes**?
 - Protected from each other!
 - OS Protected from them
 - Processes provides memory protection
- Fundamental tradeoff between protection and efficiency
 - Communication easier *within* a process
 - Communication harder *between* processes

Single and Multithreaded Processes



- Threads encapsulate **concurrency**:
 - “Active” component
- Address spaces encapsulate **protection**:
 - “Passive” component
 - Keeps buggy programs from crashing the system
- Why have multiple threads per address space?
 - Parallelism: take advantage of actual hardware parallelism (e.g., multicore)
 - Concurrency: ease of handling I/O and other simultaneous events

Protection and Isolation

- Why Do We Need Processes ?
 - Reliability: bugs can only overwrite memory of process they are in
 - Security and privacy: malicious or compromised process can't read or write other process' data
 - (to some degree) Fairness: enforce shares of disk, CPU
- Mechanisms:
 - Address translation: address space only contains its own data
 - BUT: why can't a process change the page table pointer?
 - » Or use I/O instructions to bypass the system?
 - Hardware must support **privilege levels** (OS can do more “things” than a process)

Side Note: Mechanisms vs Policy

Mechanism

Low-level methods or protocols
that implement a needed piece of
functionality

A Brake Pedal!

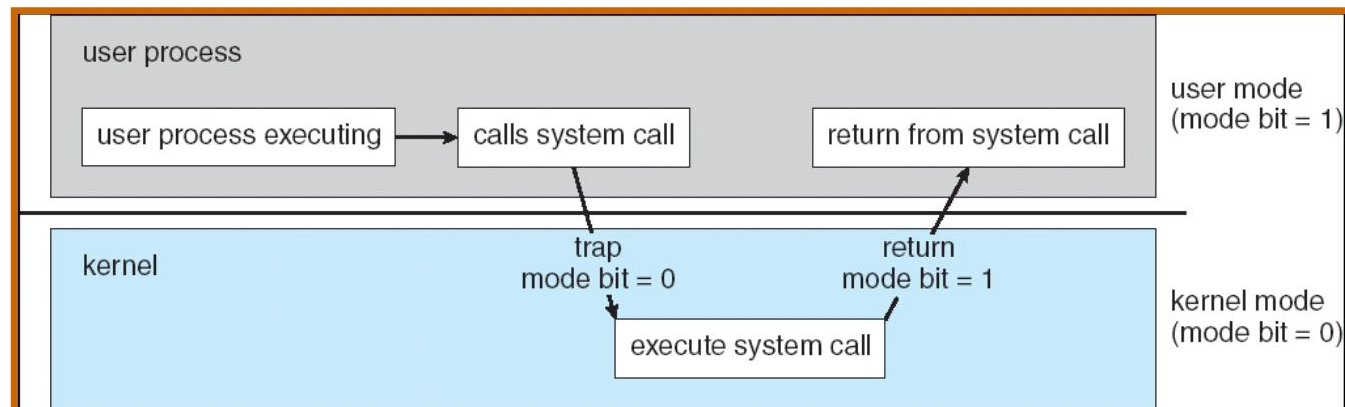
Policy

Algorithms for making decisions
within the OS.
Use the mechanism.

“I break when I see a stop sign”

Fourth OS Concept: Dual Mode Operation

- **Hardware** provides at least two modes (at least 1 mode bit):
 1. **Kernel Mode** (or “supervisor” mode)
 2. **User Mode**
- Certain operations are **prohibited** when running in user mode
 - Changing the page table pointer, disabling interrupts, interacting directly w/ hardware, writing to kernel memory
- Carefully controlled transitions between user mode and kernel mode
 - System calls, interrupts, exceptions



Conclusion: Four Fundamental OS Concepts

- **Process:** an instance of a running program
 - Protected Address Space + One or more Threads
- **Thread: Execution Context**
 - Fully describes program state
 - Program Counter, Registers, Execution Flags, Stack
- **Address space** (with or w/o **translation**)
 - Set of memory addresses accessible to program (for read or write)
 - May be distinct from memory space of the physical machine (in which case programs operate in a virtual address space)
- **Dual mode operation / Protection**
 - Only the “system” has the ability to access certain resources